CLOUD RESOURCES INC.

AI-RUN SOS

Solutions Architecture Guide

Version 2.0 | March 2026

Classification: Confidential — Internal + Authorized Partners

Prepared for: Technical Architects, Implementation Specialists, Executive Leadership, and Finance Department

AI-RUN SOS — Solutions Architecture Guide

- AI-RUN SOS — Solutions Architecture Guide
  - Table of Contents
- 1. Executive Summary
- 2. Business Context & Value Proposition
  - Problem Statement
  - Solution
  - Competitive Moat
- 3. Financial Model & Unit Economics
  - Closure Math (1/day target)
  - Cost Structure
- 4. System Architecture Overview
  - Three-Layer Architecture
  - Monorepo Layout (Turborepo)
- 5. Technology Stack
  - Backend
  - Frontend
  - Infrastructure
- 6. Data Architecture
  - Database Statistics
  - Core Entity Relationship Map
  - Key Models (Selected Detail)
    - Submission (the core revenue entity)
    - AutoSubmitQueueItem (Epic 4)
    - MarketJob (job board aggregation)
  - All 28 Enums
- 7. API Surface
  - Complete Endpoint Catalog (23 Controllers, 160+ Endpoints)
    - Authentication (no auth required)
    - Command Center (MANAGEMENT, RECRUITMENT, SALES)
    - Auto-Submit Queue (MANAGEMENT, RECRUITMENT)
    - Submissions (MANAGEMENT, RECRUITMENT, SALES)
    - Resume Formatter (MANAGEMENT, RECRUITMENT)

- Jobs (MANAGEMENT, RECRUITMENT, SALES)
- Vendors
- Consultants
- Vendor Trust
- Market Jobs
- Email Intelligence
- Analytics (30+ endpoints)
- AI Agents
- Additional Controllers

8. Authentication & Authorization
   - JWT Authentication Flow
   - RBAC Permission Matrix
9. Agent Runtime Framework
   - Architecture
   - 12 AI Agent Specifications
   - Agent Scoring Algorithms
     - MarginGuard — Margin Computation
     - TrustGraph — Trust Score Events
     - AutoSubmit — Match Scoring
10. Email Intelligence Pipeline
    - End-to-End Flow
    - Extraction Algorithms
      - Email Classifier (Priority Order)
      - Req Signal Extractor
11. Execution Engine (Epics 1-5)
    - Epic 1: Graph Outbound + Threading
    - Epic 2: Follow-up Dispatcher
    - Epic 3: Response Detection
    - Epic 4: Auto-Submit Queue
    - Epic 5: Resume Formatter
12. Frontend Architecture
    - 22 Pages (Next.js 14 App Router)
    - State Management
13. Infrastructure & DevOps
    - Docker Compose (Development)
    - Environment Variables
    - Startup Commands
14. Security Model
    - Multi-Tenant Isolation
    - Agent Security (6 Principles)

API Security
15. Call Flow Diagrams
Flow 1: Complete Submission Lifecycle
Flow 2: Daily Autopilot Plan Generation
Flow 3: Email Sync + Extraction
16. Implementation Guide
For Implementation Specialists
Day 1: Environment Setup
Day 2: Data Ingestion
Day 3: Trust + Scoring
Day 4: Auto-Submit Configuration
Day 5: Production Readiness
For Technical Architects
Adding a New Agent
Adding a New API Module
Adding a New Frontend Page
17. Knowledge Transfer Checklist
For New Team Members
For Executives
For Finance

AI-RUN SOS — Solutions Architecture Guide

Version 2.0 | March 2026 Classification: Confidential — Internal + Authorized Partners

Table of Contents

1. Executive Summary
2. Business Context & Value Proposition
3. Financial Model & Unit Economics
4. System Architecture Overview
5. Technology Stack
6. Monorepo Structure
7. Data Architecture (47 Models, 28 Enums)
8. API Surface (23 Controllers, 160+ Endpoints)
9. Authentication & Authorization (RBAC)
10. Agent Runtime Framework (12 AI Agents)
11. Email Intelligence Pipeline
12. Execution Engine (Epics 1-5)
13. Frontend Architecture (22 Pages)
14. Infrastructure & DevOps
15. Security Model
16. Call Flow Diagrams
17. Implementation Guide
18. Knowledge Transfer Checklist

1. Executive Summary

AI-RUN SOS (Staffing Operating System) is an AI-native platform that automates 90-95% of C2C/W2 IT staffing operations. The system replaces manual recruiter workflows with autonomous agents that extract job requirements from email, match consultants, send submissions, follow up in-thread, detect vendor responses, and maintain a trust graph — all with human approval gates and full audit trails.

Key Metrics: - 61M+ vendor req signals extracted from 812K+ emails - 31K vendor contacts across 7.1K vendor companies - 17K consultant profiles with skill indexing - 12 AI agents with policy-bound execution - 47 database models, 28 enums - 23 API controllers with 160+ REST endpoints - 22 frontend pages with role-based dashboards

Target: 1 closure/day at $10+/hr margin = $5M/year gross margin potential

---

2. Business Context & Value Proposition

Problem Statement

IT staffing firms operate in a high-volume, low-latency market where: - Recruiters spend 80% of time on email triage, resume formatting, and follow-ups - Vendor trust is tribal knowledge that walks out the door - Speed-to-submit directly correlates with placement success - Margin leakage occurs through rate miscalculation and lack of discipline - No single system connects email intelligence → submission → follow-up → outcome

Solution

AI-RUN SOS creates a closed-loop operating system:

```
Email Inbox → Extraction → Trust Scoring → Matching → Submission → Follow-up
→ Response Detection → Trust Update
```

For Executives: Replaces 10 recruiters with 1 operator + approval queue **For Finance:** Margin guard prevents sub-$10/hr deals; rate intelligence provides market pricing **For Technical Architects:** Policy-bound agent framework with immutable audit trail **For Implementation Specialists:** Modular NestJS services with clear API contracts

Competitive Moat

1. **Email Intelligence** — 61M+ req signals extracted, not available in any ATS
2. **Trust Graph** — Vendor scoring based on actual outcomes (ghost rate, response time, rate honesty)
3. **Execution Loop** — Only system that sends submissions AND detects responses automatically
4. **Agent Governance** — Enterprise-grade: tool allowlists, rate limits, human gates, audit logs

---

3. Financial Model & Unit Economics

Closure Math (1/day target)

```
Daily Pipeline Required:
├── 20-25 quality submissions/day
|   └── 20% submission → interview rate = 4-5 interviews
|       └── 30% interview → offer rate = 1.5 offers
|           └── 80% offer → accept rate = 1 closure/day
|
Revenue Per Closure:
├── Average margin: $10-15/hr
├── Average hours/year: 2,000
├── Gross margin per placement: $20-30K
|
Annual Projection (250 working days):
├── 250 closures × $20K avg = $5M gross margin
├── Operating costs: ~$500K (3 humans + infrastructure)
└── Net margin potential: $4.5M
```

Cost Structure

Category	Monthly Cost	Notes
Infrastructure (AWS/Azure)	$2,000-5,000	PostgreSQL, Redis, compute
Microsoft Graph API	Included	Part of M365 license
RapidAPI (JSearch)	$75	50K requests/month
Apollo.io Enrichment	$99-299	Contact data
OpenAI/LLM (future)	$200-500	Classification enhancement
3 Human Operators	$25,000-35,000	President, MD, CFO
Total Monthly	**$27,000-41,000**	

4. System Architecture Overview

Three-Layer Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
|                        LAYER 3: AGENT RUNTIME                            |
|                                                                          |
|  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐|
|  |Autopilot | | Market   | | Supply   | | Margin   | | Trust    ||
|  |   GM     | |  Pulse   | |  Radar   | |  Guard   | |  Graph   ||
|  └────┬─────┘ └────┬─────┘ └────┬─────┘ └────┬─────┘ └────┬─────┘|
|       |            |           |            |           |       |
|  ┌────┴─────┐ ┌────┴─────┐ ┌────┴─────┐ ┌────┴─────┐ ┌────┴─────┐|
|  |Recruiter | |Job Intake| |Submission| | Vendor   | |Compliance||
|  |  Agent   | |  Agent   | |  Agent   | |Onboarding| |  Agent   ||
|  └──────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘|
|                                                                          |
|  ┌──────────┐ ┌──────────┐                                             |
|  |   Req    | | Followup |       Policy Engine | Tool Router          |
|  |Collector | |Scheduler |       Rate Limiter  | Audit Logger         |
|  └──────────┘ └──────────┘                                             |
├──────────────────────────────────────────────────────────────────────────┤
|                      LAYER 2: WORKFLOW ENGINE                            |
|                                                                          |
|  Job Intake → Sourcing → Submission → Interview → Offer →               |
|  Onboarding → Billing → Payroll                                         |
|                                                                          |
|  8 State Machines: Job, Consultant, Submission, Offer,                  |
|  Assignment, Timesheet, Invoice, Daily Scoreboard                       |
├──────────────────────────────────────────────────────────────────────────┤
|                     LAYER 1: SYSTEM OF RECORD                            |
|                                                                          |
|  PostgreSQL 16 (RLS) | Prisma 6 ORM | Redis 7 Cache                   |
|  47 Models | 28 Enums | Row-Level Security | Audit Logs               |
└──────────────────────────────────────────────────────────────────────────┘
```

Monorepo Layout (Turborepo)

```
ai-run-sos/
├── apps/
|   ├── api/                NestJS REST API (23 controllers, 30 services)
|   |   └── src/
|   |       ├── auth/           JWT + RBAC
|   |       ├── submissions/    Core submission lifecycle + follow-ups +
response detection
|   |       ├── auto-submit/    Auto-submit matching queue
```

```
|   |       ├── email/          Microsoft Graph email sender
|   |       ├── resume-formatter/ Pod-specific resume templates
|   |       ├── command-center/ Autopilot daily plan
|   |       ├── ai-agents/     6 strategic AI agents
|   |       ├── analytics/     30+ analytics endpoints
|   |       ├── market-jobs/   12-source job board aggregation
|   |       ├── vendor-trust/  Trust score computation
|   |       ├── mail-intel/    Email intelligence dashboard
|   |       ├── margin-guard/  Rate card + margin computation
|   |       └── [14 more modules]
|   |
|   ├── web/                Next.js 14 frontend (22 pages)
|   |   └── src/
|   |       ├── app/            App Router pages
|   |       ├── components/     Sidebar, PageHeader, etc.
|   |       └── lib/            API client, auth store
|   |
|   └── mail-sync/          Email ingestion daemon
|       └── src/
|           ├── graphClient.ts    Graph API OAuth2 auth
|           ├── syncMailbox.ts    Delta sync from mailboxes
|           ├── daemon.ts         Hourly cron daemon
|           └── extract/          5 extractors (email, vendor, consultant,
client, req)
|
├── packages/
|   ├── db/                 Prisma schema + migrations + seed
|   |   └── prisma/
|   |       └── schema.prisma   47 models, 28 enums
|   |
|   ├── contracts/          Shared Zod schemas + TypeScript types
|   |
|   └── agent-runtime/      Agent framework
|       └── src/
|           ├── agent.ts         BaseAgent abstract class
|           ├── policy-engine.ts Policy enforcement
|           ├── tool-router.ts   Tool execution + 20 built-in tools
|           ├── rate-limiter.ts  Sliding-window rate limiter
|           ├── audit-logger.ts  Immutable buffered audit log
|           ├── registry.ts      Agent singleton registry
|           └── agents/          12 agent implementations
|
├── infra/
|   ├── docker-compose.yml   PostgreSQL 16 + Redis 7
|   └── init-db.sql          Row-Level Security policies
|
└── docs/
    ├── PRD.md               Product requirements
    ├── Workflows.md         8 state machine definitions
    ├── AgentPolicies.md     Agent security + tool policies
    ├── RBAC.md              Role permissions matrix
    ├── DataModel.md         Entity relationship docs
    └── ExecutionRoadmap.md  5-epic implementation plan
```

5. Technology Stack

Backend

Component	Technology	Version	Purpose
Runtime	Node.js	25+	Server runtime
Framework	NestJS	10	Modular REST API framework
Language	TypeScript	5.x	Type-safe development
ORM	Prisma	6.19	Database access + migrations
Auth	Passport + JWT	10/4	Token-based authentication
Validation	class-validator	0.14	Request validation
Scheduling	@nestjs/schedule	6.1	Cron jobs (follow-ups, queue)
HTTP Client	Axios	1.13	External API calls
Password	bcrypt	5	Password hashing

Frontend

Component	Technology	Version	Purpose
Framework	Next.js	14.2	React server components + App Router
Language	TypeScript	5.x	Type-safe development
Styling	Tailwind CSS	3.x	Utility-first CSS
State	Zustand	5.0	Lightweight client state
UI Components	HeadlessUI	2.2	Accessible components
Icons	Heroicons	2.2	Icon system
Validation	Zod	3.23	Schema validation

Infrastructure

Component	Technology	Version	Purpose
Database	PostgreSQL	16	Primary datastore with RLS
Cache	Redis	7	In-memory cache + rate limiting

Component	Technology	Version	Purpose
Build System	Turborepo	2.x	Monorepo orchestration
Containers	Docker Compose	-	Dev environment
Email API	Microsoft Graph	v1.0	Email send/receive/sync
Job Boards	RapidAPI (JSearch)	-	External job data
Enrichment	Apollo.io	-	Contact enrichment

6. Data Architecture

Database Statistics

Metric	Count
Prisma Models	47
Enums	28
Raw SQL Tables (additional)	12
Total Tables	~59
Composite Indexes	60+
Unique Constraints	15+
Foreign Key Relations	40+

Core Entity Relationship Map

```
Tenant (1)
├── Users (N)
├── Vendors (N)
|   ├── VendorContacts (N)
|   ├── Jobs (N)
|   |   ├── Submissions (N)
|   |   |   ├── ConsentRecord (1)
|   |   |   ├── Interviews (N)
|   |   |   ├── Offers (N)
|   |   |   └── SubmissionEmailThread (N)  ← NEW
|   |   ├── Placements (N)
|   |   └── Assignments (N)
|   ├── Invoices (N)
|   |   └── Payments (N)
|   └── Offers (N)
├── Consultants (N)
|   ├── WorkAuths (N)
|   ├── ResumeVersions (N)
|   ├── Submissions (N)
|   ├── Assignments (N)
|   ├── Timesheets (N)
|   ├── ImmigrationCases (N)
|   └── AutoSubmitQueueItems (N)  ← NEW
├── ClientCompanies (N)
├── RateCards (N)
├── DailyScoreboards (N)
├── AgentAuditLogs (N)
```

```
├── TrustEvents (N)
├── Notifications (N)
└── CommunicationEvents (N)
```

Key Models (Selected Detail)

Submission (the core revenue entity)

Field	Type	Purpose
id	cuid	Primary key
tenantId	String	Multi-tenant isolation
jobId	FK → Job	The position
consultantId	FK → Consultant	The candidate
status	Enum	DRAFT → CONSENT_PENDING → SUBMITTED → INTERVIEWING → OFFERED → ACCEPTED → REJECTED → WITHDRAWN → CLOSED
submitterType	Enum	USER or AGENT
marginApproved	Boolean	MarginGuard check passed
sentEmailId	String	Graph message ID (Epic 1)
sentConversationId	String	Thread ID for follow-ups
sentAt	DateTime	Actual send timestamp
sentTo	String	Vendor contact email
sentFrom	String	Sender mailbox

AutoSubmitQueueItem (Epic 4)

Field	Type	Purpose
reqSignalId	String?	Source: email req
marketJobId	String?	Source: job board
consultantId	FK → Consultant	Matched candidate
matchScore	Float	0-100 matching score
matchReasons	JSON	["4 skills matched", "Trusted vendor (85)"]
marginEstimate	Float?	Estimated $/hr margin
status	Enum	QUEUED → APPROVED → SENT / REJECTED / EXPIRED
expiresAt	DateTime	Auto-expire after 24h

MarketJob (job board aggregation)

Field	Type	Purpose
source	Enum	JSEARCH, JOOBLE, ADZUNA, CORPTOCORP, DICE, + 7 more
realnessScore	Int	0-100 job legitimacy score
actionabilityScore	Int	0-100 actionability score
urlStatus	Enum	ALIVE, DEAD, REDIRECT, UNKNOWN
fingerprint	String	Deduplication hash

All 28 Enums

Enum	Values
Pod	SWE, CLOUD_DEVOPS, DATA, CYBER
UserRole	MANAGEMENT, CONSULTANT, RECRUITMENT, SALES, HR, IMMIGRATION, ACCOUNTS, SUPERADMIN
SubmissionStatus	DRAFT, CONSENT_PENDING, SUBMITTED, INTERVIEWING, OFFERED, ACCEPTED, REJECTED, WITHDRAWN, CLOSED
AutoSubmitStatus	QUEUED, APPROVED, REJECTED, SENT, EXPIRED
JobStatus	NEW, QUALIFYING, ACTIVE, ON_HOLD, FILLED, CANCELLED
ConsultantReadiness	NEW, DOCS_PENDING, VERIFIED, SUBMISSION_READY, ON_ASSIGNMENT, OFFBOARDED
OfferStatus	EXTENDED, ACCEPTED, DECLINED, EXPIRED, WITHDRAWN
AssignmentStatus	ONBOARDING, ACTIVE, ENDING, COMPLETED, TERMINATED
TimesheetStatus	DRAFT, SUBMITTED, APPROVED, REJECTED, INVOICED
InvoiceStatus	DRAFT, SENT, PAID, PARTIAL, OVERDUE, DISPUTED
PaymentStatus	PENDING, COMPLETED, FAILED, REVERSED
MsaStatus	PENDING, ACTIVE, EXPIRED, TERMINATED
VerificationStatus	UNVERIFIED, PARTIAL, VERIFIED, FLAGGED
WorkAuthType	USC, GC, H1B, L1, OPT, CPT, EAD, TN, OTHER
LocationType	REMOTE, HYBRID, ONSITE
RateType	HOURLY, ANNUAL

Enum	Values
ReqSourceType	EMAIL, PORTAL, MANUAL, API
SubmitterType	USER, AGENT
ConsentType	EXPLICIT, AUTO_POLICY
InterviewStatus	SCHEDULED, COMPLETED, CANCELLED, NO_SHOW
PlacementStatus	ACTIVE, COMPLETED, TERMINATED, EXTENDED
MarginEventType	PLANNED, REALIZED, ADJUSTMENT, LEAKAGE
CommChannel	EMAIL, PHONE, SMS, PORTAL, INTERNAL_NOTE
CommDirection	INBOUND, OUTBOUND, INTERNAL
MarketJobSource	JSEARCH, JOOBLE, ADZUNA, ARBEITNOW, CAREERJET, CORPTOCORP, DICE, REMOTEOK, LINKEDIN, INDEED, ZIPRECRUITER, OTHER
EmploymentType	C2C, W2, W2_1099, FULLTIME, PARTTIME, CONTRACT, UNKNOWN
MarketJobStatus	ACTIVE, STALE, EXPIRED, CONVERTED
VendorReqStatus	NEW, REVIEWED, CONVERTED, REJECTED, EXPIRED

7. API Surface

Complete Endpoint Catalog (23 Controllers, 160+ Endpoints)

Authentication (no auth required)

Method	Endpoint	Purpose
POST	/api/auth/login	JWT login (email + password)
POST	/api/auth/register	User registration
POST	/api/auth/tenant	Create tenant

Command Center (MANAGEMENT, RECRUITMENT, SALES)

Method	Endpoint	Purpose
GET	/api/command-center/autopilot-plan	Daily 3-phase plan (morning/midday/evening)
POST	/api/command-center/compute-actionability	Recompute actionability scores (MANAGEMENT)

Auto-Submit Queue (MANAGEMENT, RECRUITMENT)

Method	Endpoint	Purpose
GET	/api/auto-submit/queue	Get queue items (filter by status)
GET	/api/auto-submit/stats	7-day stats breakdown
POST	/api/auto-submit/approve	Batch approve + send submissions
POST	/api/auto-submit/reject	Batch reject items
POST	/api/auto-submit/generate	Trigger queue build (MANAGEMENT)

Submissions (MANAGEMENT, RECRUITMENT, SALES)

Method	Endpoint	Purpose
GET	/api/submissions	List all submissions
GET	/api/submissions/stats	Pipeline statistics

Method	Endpoint	Purpose
GET	/api/submissions/due-followups	Followups needing dispatch
GET	/api/submissions/:id	Single submission detail + events + followups
POST	/api/submissions	Create new submission
POST	/api/submissions/from-req-signal	Quick submit from email req signal
POST	/api/submissions/:id/consent	Record consent decision
POST	/api/submissions/:id/send	Send submission via Graph email
PATCH	/api/submissions/:id/status	Update status + feedback
DELETE	/api/submissions/:id	Delete submission

Resume Formatter (MANAGEMENT, RECRUITMENT)

Method	Endpoint	Purpose
POST	/api/resumes/format	Generate 3 resume versions (clean, branded, watermarked)
GET	/api/resumes/:id/versions	List all resume versions
GET	/api/resumes/:id/current/:type	Get current version by type

Jobs (MANAGEMENT, RECRUITMENT, SALES)

Method	Endpoint	Purpose
GET	/api/jobs	List all jobs
GET	/api/jobs/:id	Job detail
POST	/api/jobs	Create job
POST	/api/jobs/intake	AI job intake (parse raw JD)
PATCH	/api/jobs/:id	Update job
GET	/api/jobs/:id/candidates	AI candidate matching
DELETE	/api/jobs/:id	Delete job (MANAGEMENT)

Vendors

Method	Endpoint	Purpose
GET	/api/vendors	List vendors

Method	Endpoint	Purpose
GET	/api/vendors/:id	Vendor detail
POST	/api/vendors	Create vendor
PATCH	/api/vendors/:id	Update vendor
DELETE	/api/vendors/:id	Delete vendor

Consultants

Method	Endpoint	Purpose
GET	/api/consultants	List consultants
GET	/api/consultants/:id	Detail
POST	/api/consultants	Create
PATCH	/api/consultants/:id	Update
PATCH	/api/consultants/:id/consent-policy	Update consent policy
DELETE	/api/consultants/:id	Delete

Vendor Trust

Method	Endpoint	Purpose
POST	/api/vendor-trust/compute	Compute all trust scores
GET	/api/vendor-trust/top	Top vendors by trust
GET	/api/vendor-trust/distribution	Trust score distribution
GET	/api/vendor-trust/:id	Individual vendor score

Market Jobs

Method	Endpoint	Purpose
GET	/api/market-jobs	List aggregated market jobs
GET	/api/market-jobs/stats	Source distribution stats
GET	/api/market-jobs/:id	Detail
POST	/api/market-jobs/fetch-boards	Trigger job board fetch
POST	/api/market-jobs/:id/convert	Convert to internal job

Email Intelligence

Method	Endpoint	Purpose
GET	/api/mail-intel/overview	Dashboard metrics
GET	/api/mail-intel/sync-status	Mailbox sync status
GET	/api/mail-intel/vendors	Extracted vendors
GET	/api/mail-intel/vendor-contacts	Vendor contacts
GET	/api/mail-intel/consultants	Extracted consultants
GET	/api/mail-intel/clients	Extracted clients
GET	/api/mail-intel/req-signals	Requirement signals
GET	/api/mail-intel/req-signals/:id/matches	Req with consultant matches
POST	/api/mail-intel/req-signals/:id/convert	Convert to job
GET	/api/mail-intel/skills-demand	Skills demand analysis
GET	/api/mail-intel/skills-supply	Skills supply analysis
GET	/api/mail-intel/mailboxes	Managed mailboxes
POST	/api/mail-intel/mailboxes	Add mailbox
GET	/api/mail-intel/export/*	5 CSV export endpoints

Analytics (30+ endpoints)

Method	Endpoint	Purpose
GET	/api/analytics/recruiter-activity	Recruiter performance metrics
GET	/api/analytics/email-pipeline	Email processing pipeline
GET	/api/analytics/actionable-queue	Prioritized work queue
GET	/api/analytics/closure-ranked-queue	Closure-probability ranked
POST	/api/analytics/auto-assign-queue	Auto-assign to recruiters
GET	/api/analytics/vendor-feedback-loop	Vendor response analytics
GET	/api/analytics/bench-readiness	Consultant bench scores
POST	/api/analytics/rate-intelligence	Build rate benchmarks
GET	/api/analytics/rate-intelligence	Rate benchmarks by skill/location
POST	/api/analytics/train-closure-model	Train closure prediction model

Method	Endpoint	Purpose
GET	/api/analytics/live-feed	Real-time job feed
POST	/api/analytics/apollo/enrich-top	Apollo.io contact enrichment

AI Agents

Method	Endpoint	Purpose
GET	/api/ai-agents/sales-strategist	Market analysis + rate intelligence
GET	/api/ai-agents/recruiting-strategist	Bench readiness + skill gaps
GET	/api/ai-agents/job-search-analyst	Job board quality analysis
GET	/api/ai-agents/gm-strategist	GM daily briefing
GET	/api/ai-agents/managerial-coach	Operational coaching
GET	/api/ai-agents/strategy-research	Deep strategy research

Additional Controllers

Controller	Endpoints	Purpose
Offers	5	Offer lifecycle management
Timesheets	7	Timesheet CRUD + approval
Invoices	5	Invoice generation + payment tracking
Assignments	5	Assignment + onboarding
Scoreboard	3	Daily scoreboard CRUD
Dashboard	4	Role-specific dashboards
Vendor Reqs	5	Vendor requirement tracking
PST Intel	7	PST archive intelligence
Margin Guard	3	Margin computation + rate cards

8. Authentication & Authorization

JWT Authentication Flow

```
Client                          API Server
  |                                 |
  ├── POST /api/auth/login ──────>|
  |   { email, password }           |
  |                                 ├── Validate credentials (bcrypt)
  |                                 ├── Generate JWT (24h expiry)
  |<── { accessToken, user } ──────┤
  |                                 |
  ├── GET /api/submissions ──────>|
  |   Authorization: Bearer <JWT> |
  |                                 ├── JwtAuthGuard: verify token
  |                                 ├── TenantMiddleware: extract tenantId
  |                                 ├── RolesGuard: check role permission
  |<── { data } ──────────────────┤
```

RBAC Permission Matrix

Resource	MGMT	RECRUIT	SALES	ACCOUNTS	CONSULTANT	HR	IMMIGRAT
Command Center	RW	R	R	-	-	-	-
Auto-Submit Queue	RW	RW	R	-	-	-	-
Jobs	RW	RW	RW	-	-	-	-
Submissions	RW	RW	R	-	R (own)	-	-
Consultants	RW	RW	R	-	R (own)	RW	-
Vendors	RW	R	RW	R	-	-	-
Timesheets	RW	R	-	RW	RW (own)	-	-
Invoices	RW	-	-	RW	-	-	-
Trust Scores	RW	R	R	-	-	-	-
Agent Audit Logs	R	-	-	-	-	-	-
Immigration	RW	-	-	-	-	-	RW

Resource	MGMT	RECRUIT	SALES	ACCOUNTS	CONSULTANT	HR	IMMIGRAT
Compliance	RW	-	-	-	-	RW	-

9. Agent Runtime Framework

Architecture

```
┌──────────────────────────────────────────────┐
|              Agent Request                   |
|  { agentId, tool, input, reason }            |
└──────────────────┬───────────────────────────┘
                   |
          ┌────────▼────────┐
          |  POLICY ENGINE  |
          |                 |
          |  1. Tool allowlist|
          |  2. Tenant check |
          |  3. Approval gate |
          └────────┬────────┘
                   |
          ┌────────▼────────┐
          |  RATE LIMITER   |
          |                 |
          |  Per-minute window |
          |  Per-hour window |
          |  Daily window   |
          └────────┬────────┘
                   |
          ┌────────▼────────┐
          |  TOOL ROUTER    |
          |                 |
          |  Execute handler |
          |  Measure duration |
          └────────┬────────┘
                   |
          ┌────────▼────────┐
          |  AUDIT LOGGER   |
          |                 |
          |  Immutable entry |
          |  Object.freeze() |
          |  Buffered flush |
          └────────────────┘
```

12 AI Agent Specifications

#	Agent	Role	Purpose	Allowed Tools	Ra Lir
1	AutopilotGM	Boss Agent	Daily scoreboard,	All read tools + scoreboard	10

#	Agent	Role	Purpose	Allowed Tools	Ra Lir
			pipeline tracking, action plans		
2	MarketPulse	Sales Strategist	Pod focus, rate bands, vendor ranking	job.query, vendor.query, trust.read	30
3	SupplyRadar	Recruiting Strategist	Hot bench, skill gaps, nurture plans	consultant.search/read, job.query	30
4	MarginGuard	Deal Desk	Margin computation, rate approval/block	All financial tools	20
5	TrustGraph	Analytics	Trust score computation from events	trust.compute/read	20
6	Recruiter	Sourcing	Candidate search + shortlisting	consultant.search/shortlist/read	30
7	JobIntake	Job Ops	JD parsing → structured requirements	job.create/update, trust.read	40
8	Submission	Submissions	Consent check + dedup + create	submission., *consent.*	30
9	VendorOnboarding	Vendor Ops	Vendor setup + doc collection	vendor., *document.*, email.*	20
10	Compliance	Compliance	Doc verification + blocking	compliance., *document.*	20
11	ReqCollector	JD Intake Swarm	Bulk JD parsing + pod assignment	job.create/update/query	60
12	FollowupScheduler	Follow-ups	Follow-up sequences + interview prep	email., *submission.*	30

Agent Scoring Algorithms

MarginGuard — Margin Computation

```
grossMarginHr = billRate - payRate
burdenCost    = payRate × burdenPct
payrollTax    = payRate × payrollTaxPct
vendorCut     = billRate × vendorCutPct
portalFee     = billRate × portalFeePct
netMarginHr   = grossMarginHr - burdenCost - payrollTax - vendorCut -
portalFee - otherFees

If netMarginHr >= $10/hr  → APPROVE
If netMarginHr >= $5/hr   → SOFT BLOCK (warning, override available)
If netMarginHr < $5/hr    → HARD BLOCK (CFO escalation required)
```

TrustGraph — Trust Score Events

Event	Impact
submission_accepted	+0.05
placement_completed	+0.08
on_time_payment	+0.03
positive_feedback	+0.02
interview_granted	+0.04
late_payment	-0.05
compliance_violation	-0.15
ghosting	-0.10
contract_dispute	-0.12
poor_feedback	-0.07

AutoSubmit — Match Scoring

Factor	Max Points	Logic
Skill Overlap	60	4+ skills = 60, 3 = 45, 2 = 30, 1 = 15
Vendor Trust	15	Score ≥ 70 = 15, ≥ 50 = 8
Source Quality	10	JSearch = 10, CorpToCorp = 8
Readiness	10	ON_BENCH/READY = 10
Rate Fit	5	Req rate ≥ consultant + $10 = 5
Total	**100**	Min threshold: 30

10. Email Intelligence Pipeline

End-to-End Flow

```
Microsoft 365 Mailboxes (16 accounts)
         |
         ▼
┌──────────────────────────┐
| GRAPH API DELTA SYNC     |  (daemon.ts - every 60 min)
| OAuth2 client_credentials|
| Delta tokens per folder  |
| 429 throttle handling    |
└──────────┬───────────────┘
           |
           ▼
┌──────────────────────────┐
| RAW EMAIL STORAGE        |  raw_email_message (812K+ rows)
| body_text, body_preview  |
| from_email, to_emails[]  |
| sent_at, folder          |
└──────────┬───────────────┘
           |
           ▼
┌──────────────────────────┐
| EMAIL CLASSIFIER         |  emailClassifier.ts
| 8 categories:            |
| VENDOR_REQ, CONSULTANT,  |
| CLIENT, INTERNAL,        |
| SYSTEM, PERSONAL, OTHER  |
| Rule-based (50+ rules)   |
└──────────┬───────────────┘
           |
     ┌─────┴──────┬──────────┐
     ▼     ▼      ▼          ▼
┌────────┐ ┌──────┐ ┌────────┐ ┌───────┐
|VENDOR  ||CONS.||CLIENT  ||REQ    |
|Extract ||Ext. ||Extract ||SIGNAL |
|        ||     ||        ||Extract|
|7.1K co.||17K  ||308 co. ||61M+   |
|31K cont||prof.||696 cont||signals|
└────────┘ └──────┘ └────────┘ └───────┘
```

Extraction Algorithms

Email Classifier (Priority Order)

1. Own domain → INTERNAL
2. System/no-reply → SYSTEM
3. Free email + resume patterns → CONSULTANT
4. VMS domains (vectorvms, fieldglass) → CLIENT
5. Client subject patterns (interview, PO, timesheet) → CLIENT
6. Resume score ≥ 2 → CONSULTANT
7. Req score ≥ 2 → VENDOR_REQ
8. Company domain → VENDOR_OTHER

Req Signal Extractor

- Subject patterns: c2c, w2, requirement, hiring, hot req
- Body patterns: location:, duration:, rate:, client:, $/hr (needs ≥2 hits)
- Extracts: title, location, rate text, employment type, skills
- Links to vendor_company + vendor_contact by domain

11. Execution Engine (Epics 1-5)

Epic 1: Graph Outbound + Threading

Service: `EmailSenderService` (`apps/api/src/email/email-sender.service.ts`)

```
SubmissionsService.send()
    |
    ├── Generate email draft (to, subject, body)
    |
    ├── EmailSenderService.sendMail()
    |   ├── getAccessToken() (OAuth2 client_credentials, cached)
    |   ├── POST /users/{from}/messages (create draft)
    |   |   └── Returns: id, conversationId, internetMessageId
    |   ├── POST /users/{from}/messages/{id}/send
    |   └── Retry with exponential backoff on 429/503/504
    |
    ├── Store sent evidence on Submission:
    |   sentEmailId, sentConversationId, sentAt, sentTo, sentFrom,
sentSubject
    |
    ├── Create SubmissionEmailThread entry (OUTBOUND, INITIAL)
    |
    ├── Log submission_event (SENT)
    |
    └── Schedule follow-ups: T+4h, T+24h, T+48h
```

Epic 2: Follow-up Dispatcher

Service: `FollowupDispatcherService` (cron: every 10 minutes)

```
Cron fires every 10 minutes
    |
    ├── Query: submission_followup WHERE status='PENDING' AND scheduled_at <=
NOW()
    |   JOIN Submission, Consultant, Job, Vendor, vendor_trust_score
    |
    ├── For each due follow-up:
    |   ├── IF follow-up #3+ AND vendor trust ≥ 70
    |   |   └── ESCALATE to human (don't spam high-value vendors)
    |   |
    |   ├── IF no original Graph message ID
    |   |   └── Mark as sent (manual follow-up needed)
    |   |
    |   └── ELSE:
    |       ├── EmailSenderService.replyToMessage() (in-thread reply)
```

```
    |       ├── Create SubmissionEmailThread entry
    |       └── Mark follow-up as SENT
    |
    └── Log: "N sent, N failed, N escalated"
```

Epic 3: Response Detection

Service: `ResponseDetectorService` (cron: every 5 minutes)

```
Cron fires every 5 minutes
    |
    ├── Query: raw_email_message WHERE sent_at >= NOW()-24h
    |   AND category IN (vendor_req, vendor_reply, general)
    |   AND NOT already processed for response detection
    |
    ├── For each email:
    |   ├── MATCH to submission:
    |   |   ├── Strategy 1: conversationId match (reliable)
    |   |   └── Strategy 2: subject + sender domain match (fallback)
    |   |
    |   ├── CLASSIFY response (30+ regex patterns):
    |   |   ├── INTERVIEW_REQUEST (0.9 confidence) → INTERVIEWING
    |   |   ├── NEED_RTR (0.85)
    |   |   ├── CLIENT_SUBMITTED (0.85)
    |   |   ├── RATE_TOO_HIGH (0.8)
    |   |   ├── NOT_A_MATCH (0.75) → REJECTED
    |   |   ├── POSITION_CLOSED (0.9) → CLOSED
    |   |   ├── NO_THIRD_PARTY (0.95) → REJECTED
    |   |   └── ACKNOWLEDGED (0.6)
    |   |
    |   ├── AUTO-UPDATE submission status (if confidence ≥ 0.8)
    |   ├── Cancel pending follow-ups on terminal status
    |   └── EMIT vendor_feedback_event (trust loop)
```

Epic 4: Auto-Submit Queue

Service: `AutoSubmitService` (cron: 7 AM weekdays + hourly expiry)

```
Daily at 7 AM (weekdays):
    |
    ├── Fetch premium reqs (last 48h):
    |   ├── Email reqs: C2C, actionability ≥ 50, trust ≥ 40, no spam
    |   └── Market jobs: JSEARCH + CORPTOCORP, actionability ≥ 50
    |
    ├── Fetch available consultants (READY/ON_BENCH/AVAILABLE, limit 200)
    |
    ├── For each req, score all consultants:
    |   ├── Skill overlap (0-60 pts)
    |   ├── Vendor trust bonus (0-15 pts)
    |   ├── Source quality bonus (0-10 pts)
```

```
    |   ├── Readiness bonus (0-10 pts)
    |   └── Rate fit bonus (0-5 pts)
    |
    ├── Take top 3 matches per req (score ≥ 30)
    ├── Dedup against last 7 days
    ├── Cap at 50 items/day
    └── Create AutoSubmitQueueItem (QUEUED, expires 24h)

On Approve (batch):
    |
    ├── Create Submission (via SubmissionsService)
    ├── Send via Graph (SubmissionsService.send())
    ├── Schedule follow-ups (T+4h, T+24h, T+48h)
    └── Mark queue item as SENT
```

Epic 5: Resume Formatter

Service: `ResumeFormatterService`

```
POST /api/resumes/format { consultantId, rawContent, pod }
    |
    ├── Look up consultant
    ├── Select pod template (SWE=blue, CLOUD=green, DATA=purple, CYBER=red)
    |
    ├── Generate 3 versions:
    |   ├── CLEAN: strip address, keep phone
    |   ├── VENDOR_BRANDED: strip address + add branding header
    |   └── WATERMARKED: strip address + invisible tracking code
    |
    ├── For each version:
    |   ├── Apply regex PII stripping (address, optional phone)
    |   ├── Build HTML with pod-colored header, skill tags
    |   ├── SHA-256 hash for dedup
    |   ├── Deactivate previous versions (isCurrent = false)
    |   └── Store in ResumeVersion table
    |
    └── Return { clean, vendorBranded, watermarked }
```

---

12. Frontend Architecture

22 Pages (Next.js 14 App Router)

#	Route	Purpose	Audience
1	/login	Authentication	All
2	/dashboard/command-center	Autopilot daily plan	Management
3	/dashboard/auto-submit	1-click submit queue	Management, Recruitment
4	/dashboard/closure-engine	Closure pipeline	Management
5	/dashboard/work-queue	Recruiter work queue	Management, Recruitment
6	/dashboard/ai-agents	6 AI agent dashboards	Management
7	/dashboard/recruiter-analytics	Recruiter performance	Management
8	/dashboard/live-feed	Real-time job feed	All
9	/dashboard/jobs	Job management	Recruitment, Sales
10	/dashboard/jobs/[id]	Job detail	Recruitment, Sales
11	/dashboard/vendor-reqs	Vendor requirements	Recruitment, Sales
12	/dashboard/market-jobs	Job board aggregation	Recruitment, Sales
13	/dashboard/mail-intel	Email intelligence	Management
14	/dashboard/consultants	Consultant profiles	Recruitment, HR
15	/dashboard/submissions	Submission pipeline	Recruitment
16	/dashboard/timesheets	Timesheet management	Accounts
17	/dashboard/sales	Sales dashboard	Sales
18	/dashboard/recruitment	Recruitment dashboard	Recruitment
19	/dashboard/accounts	Accounts dashboard	Accounts
20	/dashboard/pst-intel	PST archive intelligence	Management

State Management

- **Auth:** Zustand store (`useAuthStore`) with `persist` middleware → `localStorage` key `sos-auth`

- **API Client:** Centralized `api` module wrapping `fetch` with auto JWT injection, 401 redirect, error handling
- **Page State:** Local `useState` per page — no global state pollution

---

13. Infrastructure & DevOps

Docker Compose (Development)

```yaml
services:
  postgres:
    image: postgres:16
    ports: ["5432:5432"]
    environment:
      POSTGRES_DB: ai_run_sos
      POSTGRES_USER: sos_admin
      POSTGRES_PASSWORD: sos_dev_password
    volumes:
      - ./init-db.sql:/docker-entrypoint-initdb.d/init-db.sql

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
```

Environment Variables

Variable	Purpose	Example
DATABASE_URL	PostgreSQL connection	postgresql://sos_admin:... @localhost:5432/ai_run_sos
JWT_SECRET	JWT signing key	(random 64-byte string)
JWT_EXPIRES_IN	Token TTL	24h
REDIS_URL	Redis connection	redis://localhost:6379
GRAPH_TENANT_ID	Azure AD tenant	(GUID)
GRAPH_CLIENT_ID	App registration ID	(GUID)
GRAPH_CLIENT_SECRET	App secret	(secret value)
RAPIDAPI_KEY	JSearch API key	(key)
APOLLO_SEARCH_KEY	Apollo search API	(key)
APOLLO_ENRICH_KEY	Apollo enrichment API	(key)
SUBMISSION_SENDER_EMAIL	Outbound mailbox	recruiter@cloudresources.net

Startup Commands

```
# Start infrastructure
cd infra && docker-compose up -d

# Install dependencies
npm install

# Generate Prisma client
npm run db:generate

# Run migrations
npm run db:migrate

# Seed database
npm run db:seed

# Start both API + Web
npm run dev

# Start mail sync daemon (separate terminal)
npm run mail:daemon
```

14. Security Model

Multi-Tenant Isolation

- Every table has `tenantId` column
- PostgreSQL Row-Level Security (RLS) policies in `init-db.sql`
- Prisma queries automatically scoped by tenant
- JWT contains `tenantId` claim
- `TenantMiddleware` extracts and validates on every request

Agent Security (6 Principles)

1. **Disclosure:** All agent communications identify as AI
2. **Least Privilege:** Tool allowlists per agent (no agent has all tools)
3. **Audit Trail:** Every tool call logged with input, output, reason, duration
4. **Human Gates:** Immigration, payroll, contracts require human sign-off
5. **Rate Limits:** Per-minute, per-hour, daily windows per agent
6. **Sandboxing:** Agents cannot access other tenants' data

API Security

- Global `ValidationPipe` with `whitelist: true, forbidNonWhitelisted: true`
- CORS enabled
- JWT tokens expire in 24h
- bcrypt password hashing (cost factor 10)
- No secrets in client-side code

---

15. Call Flow Diagrams

Flow 1: Complete Submission Lifecycle

```
User clicks "Submit" in Auto-Submit Queue
    |
    ▼
POST /api/auto-submit/approve { itemIds: ["abc123"] }
    |
    ├── [Auth] JwtAuthGuard → validate JWT
    ├── [Auth] TenantMiddleware → extract tenantId
    ├── [Auth] RolesGuard → verify MANAGEMENT or RECRUITMENT
    |
    ├── AutoSubmitService.batchApprove()
    |   |
    |   ├── Load AutoSubmitQueueItem
    |   |
    |   ├── IF email req signal:
    |   |   └── SubmissionsService.createFromReqSignal()
    |   |       ├── Query vendor_req_signal + vendor_company + vendor_contact
    |   |       ├── Find or create Vendor in Prisma
    |   |       ├── Create Job from req signal data
    |   |       └── Call SubmissionsService.create()
    |   |
    |   ├── SubmissionsService.create()
    |   |   ├── Validate job exists + consultant exists
    |   |   ├── GUARD: Check duplicate submission
    |   |   ├── GUARD: C2C/W2 mismatch detection
    |   |   ├── GUARD: MarginGuard.checkSubmission()
    |   |   ├── Create Submission (status: DRAFT)
    |   |   ├── Log submission_event: CREATED
    |   |   ├── Evaluate consent policy
    |   |   |   ├── AUTO_APPROVED → status: SUBMITTED
    |   |   |   ├── BLOCKED → status: WITHDRAWN
    |   |   |   └── NEEDS_CONSENT → status: CONSENT_PENDING
    |   |   └── Return submission
    |   |
    |   ├── SubmissionsService.send()
    |   |   ├── Generate email draft (to, subject, body)
    |   |   ├── EmailSenderService.sendMail()
    |   |   |   ├── OAuth2 → get Graph API token
    |   |   |   ├── POST /users/{from}/messages → create draft
    |   |   |   ├── POST /users/{from}/messages/{id}/send
    |   |   |   └── Return { messageId, conversationId }
    |   |   ├── Update Submission: sentEmailId, sentAt, sentTo, ...
    |   |   ├── Create SubmissionEmailThread (OUTBOUND, INITIAL)
    |   |   ├── Log submission_event: SENT
    |   |   └── Schedule follow-ups: T+4h, T+24h, T+48h
    |   |
    |   └── Update AutoSubmitQueueItem: status=SENT, submissionId
```

```
    |
    └── Return results to frontend

---[LATER: T+4 hours]---

FollowupDispatcherService (cron every 10 min)
    ├── Find due follow-ups
    ├── EmailSenderService.replyToMessage() → in-thread reply
    └── Mark follow-up SENT

---[LATER: Vendor replies]---

ResponseDetectorService (cron every 5 min)
    ├── Scan raw_email_message for new inbound
    ├── Match to submission by conversationId
    ├── Classify: "INTERVIEW_REQUEST" (confidence 0.9)
    ├── Auto-update submission: SUBMITTED → INTERVIEWING
    ├── Cancel remaining follow-ups
    └── Emit vendor_feedback_event: INTERVIEW_GRANTED
```

Flow 2: Daily Autopilot Plan Generation

```
GET /api/command-center/autopilot-plan
    |
    ├── CommandCenterService.getAutopilotPlan()
    |   |
    |   ├── [Cache Check] 2-minute TTL in-memory cache
    |   |
    |   ├── PARALLEL QUERIES:
    |   |   ├── getActionableReqs(50)
    |   |   |   ├── Query vendor_req_signal (C2C, last 48h, trust ≥ 40)
    |   |   |   ├── Query MarketJob (JSEARCH + CORPTOCORP, last 3 days)
    |   |   |   └── Sort: JSearch(1) > CorpToCorp(2) > Email(3), then
createdAt DESC
    |   |   |
    |   |   ├── getBenchMatchReqs(20)
    |   |   |   └── Reqs with matching ON_BENCH consultants
    |   |   |
    |   |   ├── getDueFollowups()
    |   |   |   └── submission_followup WHERE status=PENDING AND scheduled_at
<= NOW()
    |   |   |
    |   |   ├── getStuckSubmissions()
    |   |   |   └── Submissions with status=SUBMITTED for >48h, no response
    |   |   |
    |   |   ├── getTodayActivity()
    |   |   |   └── Submissions, interviews, offers created today
    |   |   |
    |   |   └── getVendorLeaderboard(10)
    |   |       └── Top 10 vendors by trust_score
    |   |
    |   └── Assemble 3-phase plan:
    |       ├── Morning (9 AM): actionableReqs + benchMatches +
submissionQuota
```

```
    |       ├── Midday (2 PM): followupsDue + stuckSubmissions
    |       └── Evening (6 PM): todayActivity + closureProbability +
vendorLeaderboard
    |
    └── Return plan to Command Center UI
```

Flow 3: Email Sync + Extraction

```
mail:daemon (cron every 60 min)
    |
    ├── validateCredentials()
    |   └── OAuth2 client_credentials → Graph API token
    |
    ├── Query mailbox table → 16 email accounts
    |   └── Exclude: akkayya.chavala@, accounts@, info@
    |
    ├── For each mailbox:
    |   └── syncMailbox(email)
    |       ├── discoverFolders() → list all mail folders
    |       ├── Filter out: Conversation History, RSS, Outbox
    |       ├── For each folder:
    |       |   └── syncFolder()
    |       |       ├── GET /users/{email}/mailFolders/{id}/messages?$top=50
    |       |       ├── INSERT INTO raw_email_message ON CONFLICT DO NOTHING
    |       |       ├── Stop after 3 consecutive pages with 0 new inserts
    |       |       └── Handle 429 (wait retry-after) + 401 (refresh token)
    |       └── Update mailbox.last_synced_at
    |
    ├── IF any new emails found:
    |   ├── classifyNewEmails()      → categorize by 50+ rules
    |   ├── extractVendors()         → upsert vendor_company + vendor_contact
    |   ├── extractConsultants()     → upsert consultant (skills, resume
detection)
    |   ├── extractClients()         → upsert client_company + client_contact
    |   └── extractReqSignals()      → create vendor_req_signal entries
    |
    └── Log totals
```

16. Implementation Guide

For Implementation Specialists

Day 1: Environment Setup

1. Clone repository, run `npm install`
2. Start Docker (PostgreSQL + Redis): `cd infra && docker-compose up -d`
3. Copy `.env` and configure Graph API credentials
4. Run `npm run db:generate && npm run db:migrate && npm run db:seed`
5. Start with `npm run dev` — API on :3001, Web on :3000
6. Login: md@apex-staffing.com / Password123!

Day 2: Data Ingestion

1. Add mailboxes via `/api/mail-intel/mailboxes`
2. Run initial sync: `npm run mail:sync`
3. Run extraction: `npm run mail:extract`
4. Verify data: check vendor/consultant counts on Email Intel page

Day 3: Trust + Scoring

1. Trigger trust computation: `POST /api/vendor-trust/compute`
2. Trigger actionability scoring: `POST /api/command-center/compute-actionability`
3. Verify Command Center shows actionable reqs

Day 4: Auto-Submit Configuration

1. Ensure consultants have readiness = READY or ON_BENCH
2. Trigger queue build: `POST /api/auto-submit/generate`
3. Review queue on Auto-Submit page
4. Test approve flow (small batch)

Day 5: Production Readiness

1. Configure `SUBMISSION_SENDER_EMAIL` for actual mailbox
2. Start mail daemon: `npm run mail:daemon`
3. Verify follow-up dispatcher is running (check logs for “No due follow-ups”)
4. Verify response detector is running (check logs for “Scanning N inbound emails”)

For Technical Architects

Adding a New Agent

1. Create `my-agent.agent.ts` in `packages/agent-runtime/src/agents/`
2. Extend `BaseAgent`, implement `execute(context: AgentContext)`
3. Define `allowedTools`, `approvalRequired`, `rateLimits` in config
4. Register in `packages/agent-runtime/src/index.ts`
5. Use `this.callTool()` for all tool invocations (policy + rate limit + audit)
6. Handle escalation with `this.escalate()`

Adding a New API Module

1. Create folder in `apps/api/src/my-module/`
2. Create: `my-module.service.ts`, `my-module.controller.ts`, `my-module.module.ts`
3. Register in `apps/api/src/app.module.ts`
4. Use `@UseGuards(JwtAuthGuard, RolesGuard)` + `@Roles(...)` for auth
5. Inject `PrismaService` for database access

Adding a New Frontend Page

1. Create `apps/web/src/app/dashboard/my-page/page.tsx`
2. Use `'use client'` directive
3. Import `api` from `@/lib/api` for data fetching
4. Add nav item in `apps/web/src/components/sidebar.tsx` per role

---

17. Knowledge Transfer Checklist

For New Team Members

- [ ] Read this Solutions Guide end-to-end
- [ ] Read `docs/PRD.md` for product vision
- [ ] Read `docs/Workflows.md` for state machine definitions
- [ ] Read `docs/AgentPolicies.md` for agent security model
- [ ] Read `docs/RBAC.md` for role permissions
- [ ] Set up local environment (see Implementation Guide Day 1)
- [ ] Login and explore all dashboard pages
- [ ] Trace a submission through the system (create → send → follow-up)
- [ ] Review `apps/api/src/submissions/submissions.service.ts` (core business logic)
- [ ] Review `apps/api/src/email/email-sender.service.ts` (Graph integration)
- [ ] Review `packages/agent-runtime/src/agent.ts` (agent framework)

For Executives

- [ ] Understand the 1-closure/day math (Section 3)
- [ ] Review the competitive moat (Section 2)
- [ ] Understand the trust graph value prop (vendor scoring from real outcomes)
- [ ] Review the API surface breadth (160+ endpoints = mature platform)
- [ ] Understand the execution loop (email → send → follow-up → detect → trust update)

For Finance

- [ ] Review unit economics (Section 3)
- [ ] Understand MarginGuard ($10/hr floor with $5/hr hard block)
- [ ] Review rate intelligence capabilities (`/api/analytics/rate-intelligence`)
- [ ] Understand timesheet → invoice → payment flow
- [ ] Review margin event tracking (planned vs. realized vs. leakage)

Document generated: March 2026 System version: 2.0 (Post-Execution Engine) Total codebase: ~50,000 lines TypeScript